

INGRESS

- Ingress is a Kubernetes resource that manages external access to services using a single Load Balancer.
- It routes incoming HTTP/HTTPS requests to different services based on host names or URL paths.
- Instead of creating multiple LoadBalancers, Ingress uses one Load Balancer to serve multiple applications.

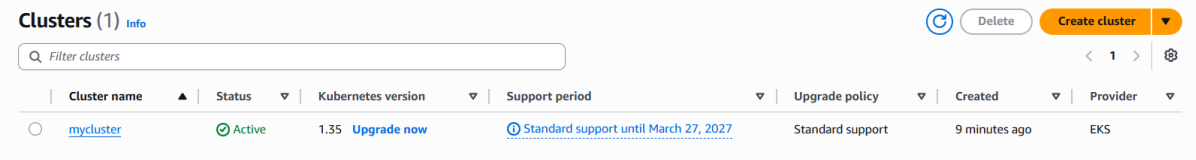
Uses of Ingress

- Expose multiple services using a single Load Balancer.
- Route traffic based on domain names (Host-based routing).
- Route traffic based on URL paths (Path-based routing).
- Enable HTTPS/SSL for applications.
- Reduce the number of Load Balancers required.

HANDS-ON FOR INGRESS

Screenshot: Create EKS Cluster

- Open the **AWS Management Console**.
- Navigate to **Amazon EKS**.
- Click **Create Cluster**.
- Enter the cluster name.
- Configure the required settings.
- Click **Create**.
- EKS Cluster created successfully.



Clusters (1) Info								Delete	Create cluster	
Filter clusters										
	Cluster name	Status	Kubernetes version	Support period	Upgrade policy	Created	Provider			
☐	mycluster	 Active	1.35 Upgrade now	Standard support until March 27, 2027	Standard support	9 minutes ago	EKS			

Screenshot: Create Worker Nodes

- Create a Node Group for the EKS Cluster.
- Configure **2 Worker Nodes**.
- Select the instance type and required settings.
- Click **Create**.

- Two Worker Nodes created successfully.

Nodes (2) [Info](#)

Q

Filter Nodes by property or value

<

1

>

Node name	Instance type	Compute	Managed by	Created	Status	CPU usage	Memory usage	Ephemeral storage usage
ip-172-31-11-5.ap-south-1.compute.internal	c7i-flex.large	Node group	nodegrp	a minute ago	Ready	32%	34%	15%
ip-172-31-22-59.ap-south-1.compute.internal	c7i-flex.large	Node group	nodegrp	a minute ago	Ready	12%	19%	15%

Node groups (1) [Info](#)

Edit

Delete

Add

Node groups implement basic compute scaling through EC2 Auto Scaling groups.

Q

Filter node groups by property or value

<

1

>

Group name	Desired size	AMI release version	Launch template	Status
nodegrp	2	1.35.5-20260529	-	Active

Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elas
☒ eks-access	i-05a7963e790e51871	Running	c7i-flex.large	Initializing	ap-south-1c	ec2-3-6-116-179.ap-so...	3.6.116.179	-
☐ eks-access	i-0ce1a74d0f7e5dbc1	Pending	t3.micro	-	ap-south-1b	-	-	-
☒ eks-access	i-05f78f9c7496197c0	Running	c7i-flex.large	Initializing	ap-south-1b	ec2-15-206-165-11.ap-...	15.206.165.11	-

Screenshot: Create EC2 Instance and Connect to EKS Cluster

- Launch an EC2 instance.
- Connect to the EC2 instance using SSH.
- Run aws configure.
- Enter the AWS Access Key, Secret Key, Region, and Output Format.
- Run `aws eks update-kubeconfig --region <region-name> --name <cluster-name>`.
- EKS cluster configured successfully.
- EC2 instance connected to the EKS cluster successfully.

```
[ec2-user@ip-172-31-4-149 ~]$ aws eks --region ap-south-1 update-kubeconfig --name mycluster
Updated context arn:aws:eks:ap-south-1:301804611986:cluster/mycluster in /home/ec2-user/.kube/config
[ec2-user@ip-172-31-4-149 ~]$
```

Screenshot: Install eksctl

- Run the eksctl installation commands on the EC2 instance.
- Verify the installation using eksctl version.

```
[ec2-user@ip-172-31-4-149 ~]$
[ec2-user@ip-172-31-4-149 ~]$ # Install eksctl
ARCH=amd64
PLATFORM=$(uname -s)_$ARCH
curl -sLO "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_${PLATFORM}.tar.gz"
curl -sL "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_checksums.txt" | grep $PLATFORM | sha256sum --check
tar -xzf eksctl_${PLATFORM}.tar.gz -C /tmp && rm eksctl_${PLATFORM}.tar.gz
sudo install -m 0755 /tmp/eksctl /usr/local/bin && rm /tmp/eksctl
eksctl Linux_amd64.tar.gz: OK
[ec2-user@ip-172-31-4-149 ~]$
```

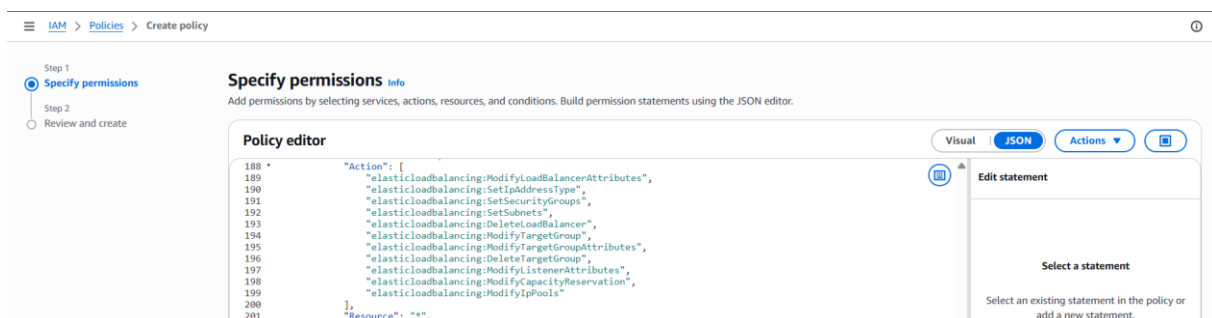
Screenshot: Associate IAM OIDC Provider

- Run `eksctl utils associate-iam-oidc-provider --region ap-south-1 --cluster mycluster --approve`.
- Press Enter.
- IAM OIDC provider associated successfully with the EKS cluster.

```
[ec2-user@ip-172-31-4-149 ~]$  
[ec2-user@ip-172-31-4-149 ~]$ eksctl utils associate-iam-oidc-provider \\  
--region=ap-south-1 \\  
--cluster=mycluster \\  
--approve  
2026-06-14 13:28:14 [i] will create IAM Open ID Connect provider for cluster "mycluster" in "ap-south-1"  
2026-06-14 13:28:15 [✓] created IAM Open ID Connect provider for cluster "mycluster" in "ap-south-1"  
[ec2-user@ip-172-31-4-149 ~]$
```

Screenshot: Create IAM Policy for AWS Load Balancer Controller

- Open the **AWS Management Console**.
- Navigate to **IAM**.
- Click **Policies**.
- Click **Create Policy**.
- Select the **JSON** tab.
- Paste the AWS Load Balancer Controller policy JSON code.
- Click **Next**.
- Enter the policy name as **AWSLoadBalancerControllerPolicy**.
- Click **Create Policy**.
- IAM Policy created successfully.



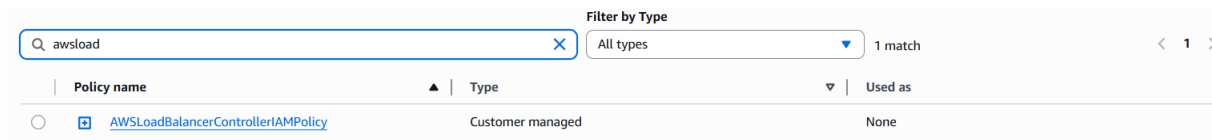
Policy details

Policy name

Enter a meaningful name to identify this policy.

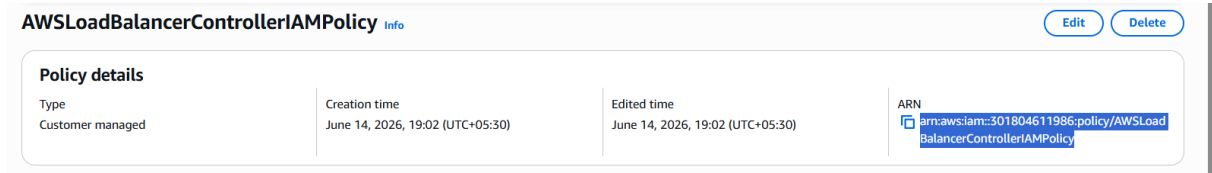
AWSLoadBalancerControllerIAMPolicy

Maximum 128 characters. Use alphanumeric and '+,=,@-_' characters.



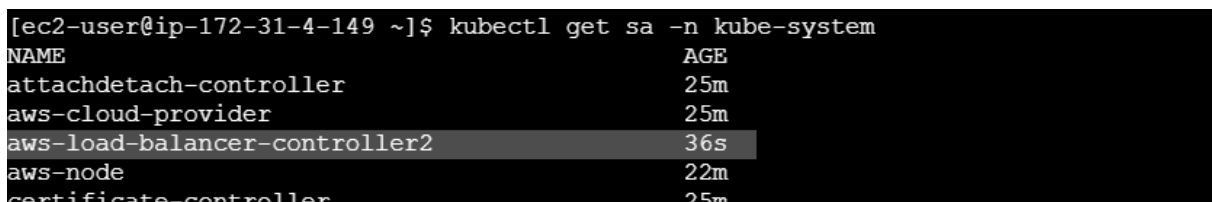
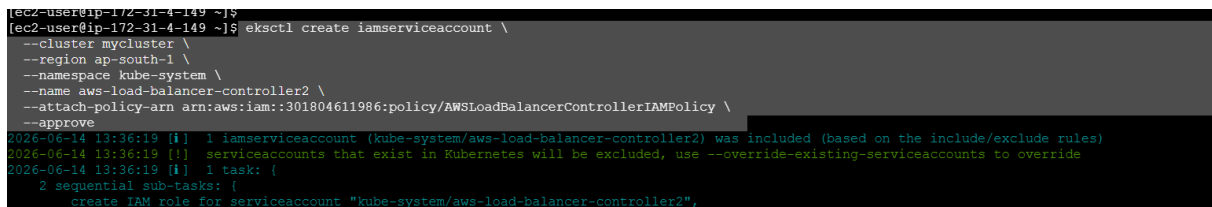
Screenshot: Copy Policy ARN

- Select **AWSLoadBalancerControllerPolicy**.
- Copy the **Policy ARN**.



Screenshot: Create IAM Service Account

- Run `eksctl create iamserviceaccount --cluster=mycluster --region=ap-south-1 --namespace=kube-system --name=aws-load-balancer-controller --attach-policy-arn=<Policy-ARN> --override-existing-serviceaccounts --approve`.
- Press Enter.
- IAM Service Account created successfully.



Screenshot: Install Helm

- Download Helm.
- Install Helm.
- Verify the installation.
- Helm installed successfully.

```
[ec2-user@ip-172-31-4-149 ~]$
[ec2-user@ip-172-31-4-149 ~]$ curl -fsSL -o get_helm.sh https://raw.githubusercontent.com/helm/helm/main/scripts/get-helm-3
chmod 700 get_helm.sh
./get_helm.sh
[WARNING] Could not find git. It is required for plugin installation.
Downloading https://get.helm.sh/helm-v3.21.1-linux-amd64.tar.gz
Verifying checksum... Done.
Preparing to install helm into /usr/local/bin
helm installed into /usr/local/bin/helm
```

Screenshot: Add EKS Helm Repository

- Add the Amazon EKS Helm repository.
- Update the Helm repositories.
- EKS Helm repository added and updated successfully.

```
[ec2-user@ip-172-31-4-149 ~]$ helm repo add eks https://aws.github.io/eks-charts
helm repo update
"eks" has been added to your repositories
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "eks" chart repository
Update Complete. *Happy Helming!*
```

Screenshot: Get VPC ID of EKS Cluster

- Run the command to retrieve the VPC ID of the EKS cluster.
- Specify the cluster name as **mycluster**.
- Specify the region as **ap-south-1**.
- VPC ID displayed successfully

```
[ec2-user@ip-172-31-4-149 ~]$ aws eks describe-cluster \
--name mycluster \
--region ap-south-1 \
--query "cluster.resourcesVpcConfig.vpcId" \
--output text
vpc-00e925f7d875b3505
[ec2-user@ip-172-31-4-149 ~]$
```

Screenshot: Install AWS Load Balancer Controller

- Install the AWS Load Balancer Controller using Helm.
- Specify the cluster name as **mycluster**.
- Specify the region as **ap-south-1**.
- Specify the VPC ID.
- Use the existing service account **aws-load-balancer-controller**.
- AWS Load Balancer Controller installed successfully.

```
[ec2-user@ip-172-31-4-149 ~]$
[ec2-user@ip-172-31-4-149 ~]$ helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName=mycluster \
  --set region=ap-south-1 \
  --set vpcId=vpc-00e925f7d875b3505 \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller2
NAME: aws-load-balancer-controller
LAST DEPLOYED: Sun Jun 14 13:43:32 2026
NAMESPACE: kube-system
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
AWS Load Balancer controller installed!
[ec2-user@ip-172-31-4-149 ~]$
```

Screenshot: Create Namespaces

- Run vi namespace.yaml.
- Paste the Namespace configuration for **namespace1** and **namespace2**.
- Press Esc.
- Type :wq!.
- Run kubectl apply -f namespace.yaml.
- Namespaces created successfully.

```
[ec2-user@ip-172-31-4-149 ~]$ vi namespace.yaml
[ec2-user@ip-172-31-4-149 ~]$
```

```
apiVersion: v1
kind: Namespace
metadata:
  name: namespace1
---
apiVersion: v1
kind: Namespace
metadata:
  name: namespace2
```

```
[ec2-user@ip-172-31-4-149 ~]$ kubectl apply -f namespace.yaml
namespace/namespace1 created
namespace/namespace2 created
```

```
namespace/namespace2 created
[ec2-user@ip-172-31-4-149 ~]$ kubectl create namespace jai
namespace/jai created
[ec2-user@ip-172-31-4-149 ~]$
```

Screenshot: Create Application and Ingress

- Run vi app1.yaml.
- Paste the Deployment, Service, and Ingress configuration.
- Press Esc.
- Type :wq!.

- Run `kubectl apply -f app1.yaml`.
- Deployment, Service, and Ingress created successfully.

```
[ec2-user@ip-172-31-4-149 ~]$
```

```
[ec2-user@ip-172-31-4-149 ~]$ vi app1.yaml
```

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: app2
  namespace: namespace2
spec:
  replicas: 2
  selector:
    matchLabels:
      app: app2
  template:
    metadata:
      labels:
        app: app2
    spec:
      containers:
        - name: app2
          image: hashicorp/http-echo
          args: ["-text=Namespace2 result"]
          ports:
            - containerPort: 5678
apiVersion: v1
kind: Service
metadata:
  name: namespace2-svc
  namespace: namespace2
spec:
  selector:
    app: app2
  ports:
    - port: 80
      targetPort: 5678
apiVersion: networking.k8s.io/v1
```

```
[ec2-user@ip-172-31-4-149 ~]$ kubectl apply -f app1.yaml
```

```
deployment.apps/app2 created
service/namespace2-svc created
Warning: annotation "kubernetes.io/ingress.class" is deprecated, please use 'spec.ingressClassName' instead
ingress.networking.k8s.io/namespace2-ingress created
```

Screenshot: Create Application and Ingress

- Run `vi app2.yaml`.
- Paste the Deployment, Service, and Ingress configuration.
- Press Esc.
- Type `:wq!`.
- Run `kubectl apply -f app2.yaml`.
- Deployment, Service, and Ingress created successfully.

```
[ec2-user@ip-172-31-4-149 ~]$ vi app2.yaml
```

```

- containerPort: 5678
---
apiVersion: v1
kind: Service
metadata:
  name: namespace1-svc
  namespace: namespace1
spec:
  selector:
    app: appl
  ports:
    - port: 80
      targetPort: 5678
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: namespace1-ingress
  namespace: namespace1
  annotations:
    kubern(paste) --gress.class: alb
    alb.ingress.kubernetes.io/scheme: internet-facing
    alb.ingress.kubernetes.io/target-type: ip
    alb.ingress.kubernetes.io/group.name: shared-alb
spec:
  rules:
    - http:
        paths:
          - path: /firstnamespace
            pathType: Prefix

```

```

[ec2-user@ip-172-31-4-149 ~]$ kubectl apply -f app2.yaml
deployment.apps/appl created
service/namespace1-svc created
Warning: annotation "kubernetes.io/ingress.class" is deprecated, please use 'spec.ingressClassName' instead
ingress.networking.k8s.io/namespace1-ingress created
[ec2-user@ip-172-31-4-149 ~]$

```

Screenshot: Verify Load Balancer

- Wait for the Ingress resources to be provisioned.
- Verify that the AWS Application Load Balancer is created.
- Confirm the Load Balancer status is **Active**.
- Load Balancer created successfully.

Load balancers (1/1) [What's new?](#) Actions Create load balancer

Elastic Load Balancing scales your load balancer capacity automatically in response to changes in incoming traffic.

Filter load balancers

☒	Name	State	Type	Scheme	IP address type	VPC ID	Availability Zones	Security groups
☒	k8s-sharedalb-2bb1818...	Provisioning...	application	Internet-facing	IPv4	vpc-00e925f7d875b3505	3 Availability Zones	2 Security groups